PROJECT REPORT

1. Introduction

--This project focuses on the design and implementation of a software system for sensor-based data acquisition, signal processing, and visualization. The primary objective is to develop a platform capable of loading time-series datasets,
applying signal processing techniques, and presenting meaningful insights through graphical representations.

----The motivation behind this project is to simulate real-world signal analysis systems commonly used in biomedical and environmental applications.
By working with datasets such as ECG, respiration, and temperature signals, the system demonstrates how raw sensor data can be transformed into useful information through preprocessing,
filtering, and frequency analysis.

--To ensure the system meets its intended goals, both functional and non-functional requirements were considered. Functional requirements define the core
capabilities of the system, such as loading CSV files, applying filters, computing statistical metrics, and displaying results. Non-functional requirements
focus on system quality, including usability, responsiveness, and robustness when handling large datasets.

--Another important aspect of the project is the selection of appropriate datasets for testing and validation. Publicly available datasets were used
to ensure realistic signal characteristics and reliable evaluation of the implemented methods.

--To verify correctness, processed signals are compared with the original raw data through graphical visualization. Time-domain plots and frequency-domain
analysis using FFT allows clear observation of noise reduction and signal enhancement.

--Finally, the system is designed with user experience in mind by implementing a simple and intuitive graphical user interface (GUI). This allows users to easily load data,
select the signal type, apply processing techniques, and interpret results without requiring advanced technical knowledge.

2.The system follows a sequential processing pipeline:

--CSV Input and Pre-processing: Handles file selection using a GUI. Extracts time and signal data from the selected CSV and handles missing and invalid data.
--Filtering: Filters data using FIR or IIR methods depending on selection. Choice of low, high, and bandpass filters. Filter parameters are determined based on signal type; 0.5-25Hz for ECG, 0.1-0.4Hz for respiration, and very low frequencies for temperature.
--FFT Analysis: Displays Fast Fourier Transform of the original and filtered signal.
--Feature Extraction: Computes statistical metrics such as mean, standard deviation, RMS, and peak-to-peak. Based on the selected signal type heart rate (ECG), breath and respiration rate (Respiration), or average and trend (temperature) values are calculated.
--Visualization: Finally the output of the previous steps is displayed. This may be in the form of a frequency domain plot for FFT or time domain plot for the filters.

Signal Processing Methodology
3.1 Preprocessing

Raw sensor data may contain missing or corrupted values. These values are handled using interpolation techniques to ensure continuity in the signal. This step is essential for accurate filtering and analysis.

3.2 Digital Filtering

Digital filters are used to improve signal quality:

--Low-Pass Filter (LPF): Removes high-frequency noise.
--High-Pass Filter (HPF): Eliminates baseline drift.
--Band-Pass Filter (BPF): Isolates the frequency range of interest.

For example:

--ECG signals are filtered between 0.5–25 Hz to isolate cardiac activity.
--Respiration signals are filtered between 0.1–0.4 Hz to capture breathing patterns.

3.3Frequency-Domain Analysis (FFT)

--The Fast Fourier Transform (FFT) is used to convert signals from the time domain to the frequency domain.
This allows identification of dominant frequency components, which are useful for detecting periodic patterns such as heart rate and breathing rate. It also helps to verify the functionality of the filters.

3.4 Feature Extraction
Features are extracted from processed signals to provide meaningful quantitative insights:

ECG: Heart rate (BPM), R-peak detection
Respiration: Breathing rate, RMS amplitude
Temperature: Mean value, variance, trend slope

These features help interpret physiological and environmental conditions from the sensor data.

4. Implementation Details

--The system is implemented using Python due to its powerful libraries for signal processing and visualization.

4.1 Tools and Libraries

--NumPy (numerical operations)
--Pandas (CSV handling)
--SciPy (signal processing)
--Matplotlib (visualization)
--Tkinter (GUI development)

4.2 GUI Design

The graphical user interface includes:

--File loading functionality
--Signal type selection
--Buttons for applying filters
--FFT visualization
--Display of statistical metrics

The GUI is designed to be simple and user-friendly, allowing users to interact with the system easily.

5. Testing & Evaluation

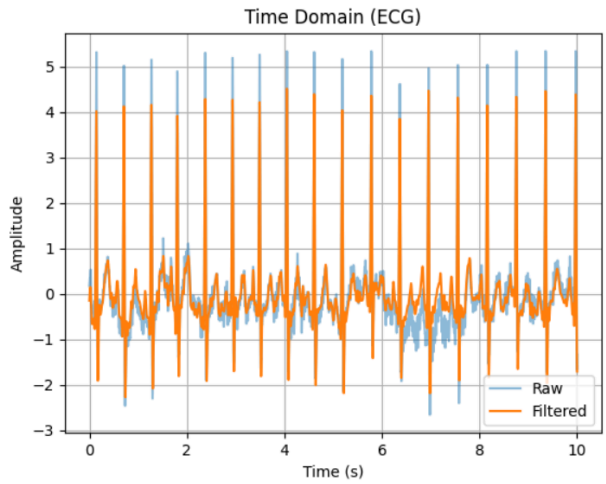
5.1 Datasets Used

The system was tested using:

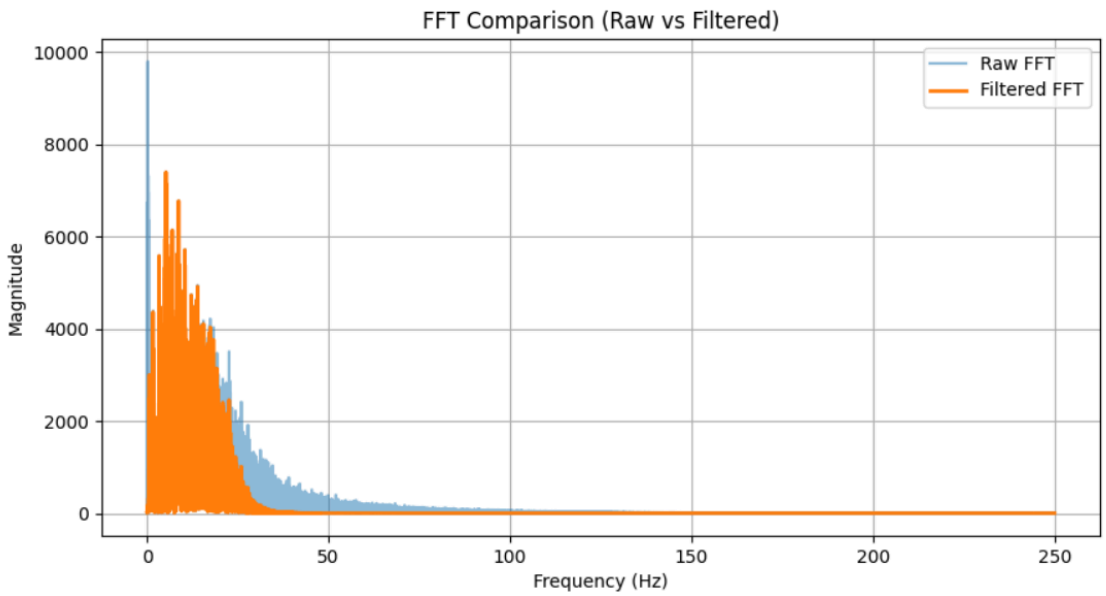
-Laboratory ECG dataset (from the course material)
--Then we switched to a publicly available experimental dataset
--Publicly available temperature dataset
--Public respiration dataset
–See section 7 for more details

5.2 Results

The following time series plot shows the original ECG signal in blue and the bandpassed output in orange. The lower cutoff of the filter worked to remove baseline drift/wander and the upper cutoff higher frequency noise but also capped the signal peaks.



The following is the FFT of the same raw and filtered signals



It's clear that the bandpass filter greatly attenuates frequency components greater than 25Hz and lesser than 0.5Hz.

The following shows the features extracted from the ECG dataset.

Mean: -0.000
STD: 0.891
RMS: 0.891
P2P: 7.713
Heart Rate: 109.89 BPM

Relevant statistical data such as the mean and STD were extracted. The mean is centered at 0 confirming that the data was properly preprocessed. The heart rate was also extracted since an ECG data set was used.

The results demonstrate that:

--Filtering effectively reduces noise
--FFT correctly identifies dominant frequencies
--Feature extraction provides meaningful insights

Unit testing was also done to verify the functionality of the system.

6 tests were run to verify the functionality of the CSV handler, filters, and feature extraction.

```
Time length: 100
Signal length: 100
Sampling frequency: 99.00000000000016
.Duration: 1.0
Samples: 50
Duration: 1.0
Time length: 50
Signal length: 50
Sampling frequency: 49.000000000000014
.Duration: 1.0
Samples: 100
Duration: 1.0
Time length: 100
Signal length: 100
Sampling frequency: 99.00000000000016
..
----------------------------------------------------------------------
Ran 6 tests in 0.038s

OK
```

The unit tests demonstrate the robustness of the system when it encounters errors or unexpected values.

5.3 Discussion

--The system successfully processes different types of signals and adapts filtering techniques accordingly.
The comparison between raw and processed signals shows significant improvement in signal clarity.

6.Conclusion

--This project successfully demonstrates the development of a complete signal processing system capable of handling sensor-based datasets.
--The system integrates data loading, preprocessing, filtering, frequency analysis, and visualization into a single platform.
--The results confirm that the implemented methods effectively improve signal quality and extract meaningful features.
--Future improvements could include real-time data processing and advanced feature extraction techniques. Other limitations include lack of filter customization. Cutoffs were predetermined for the sake of simplifying the user experience but may impede more complex analyses.

7. Data Sources

--For this project, we focused on obtaining realistic datasets collected from experimental studies. Three types of signals were required:(ECG, temperature, and respiration). Two main sources were used to obtain these datasets:

1. Kaggle – used to obtain the temperature dataset
2. PhysioNet – used to obtain the ECG and respiration datasets

--The data from PhysioNet was initially provided in WFDB (.dat) format, which required conversion to CSV format for compatibility with our system. This section was the most challenging. Many of the data sets we found were not in the CSV format but rather in WFDB. We have not yet seen this type of data in the labs or lectures. Despite following tutorials we chose to abandon the idea of using WFDB files, a different dataset from the same source was selected.

--Another challenge involved handling timestamp formats. The data used a detailed time structure in the format: year / month / day / hour / minute / second / millisecond This required careful parsing and conversion into a continuous time vector for proper signal processing.

--In addition, for the respiration datasets (ProcessedData_Subject08_FEM and ProcessedData_Subject05_FEM), column names originally contained spaces.
These were modified by replacing spaces ' ' with underscores ' _' to ensure compatibility with the implemented code, avoiding the need for additional preprocessing functions.

--The three main datasets used in this project are:
-MLTempDataset.csv
-processedData_Subject05_FEM / (processedData_Subject08_FEM[not used anymore])
-s10_run / s10_walk

Kaggle TEMP--> https://www.kaggle.com/datasets/vitthalmadane/ts-temp-1
physionet ECG-->
https://www.physionet.org/content/pulse-transit-time-ppg/1.0.0/csv/#files-panel
physionet Respiratory-->
https://physionet.org/content/respiratory-heartrate-dataset/1.0.0/subject-info.csv (this is about subject info)
https://physionet.org/content/respiratory-heartrate-dataset/1.0.0/Processed_Dataset/#files-panel (this is where we got the files--> CSV)

Data Quality Note
--An issue was identified with the Subject 08 respiration dataset, where the first few seconds of the signal contained incorrect or inconsistent values.
-As a result, this dataset was excluded from the final results to ensure accuracy and reliability of the analysis.